

Advanced C Course

Code Quality applied to C - Session 6

Pascal Rothnemer

10 March 2026, EPITA

Session 6 Content

- 1) Hands-on on Your Projects
- 2) Rehearsal exercises
- 3) Course : Multi-task & High Performance Computing
(Overview)

Schedule

- 1) **Today** – Session : Hands-on with help
- 2) **today → March 16th** – Finalization
- 3) **March 16th** : Project sent to me
- 4) March **17th** : projects presentation / evaluation
- 5) Results within 1 week

Projects Deliverables

- 1) **Source Code** with supporting readme.txt or equivalent (Compiles & Runs)
 - Comes with needed environment (files, libraries...) if any
- 2) **Presentation** and **Demo**, should discuss :
 - Project scope – reasons for choice
 - Development case : difficulties, choices, management of unexpected...
 - Description of features

Evaluation Criteria

- Attendance & participation, MCQ result
- Project
 - Features added or created
 - presentation & design (including answers to questions)
 - code quality (readability, structuration...)
 - code complexity and difficulty

Projects Hands-on

1) Hands-on on Your Projects

2) Rehearsal exercises

3) Course : Multi-task & High Performance Computing
(Overview)

Projects Hands-on

1) Hands-on on Your Projects

2) Rehearsal exercises

3) Course : Multi-task & High Performance Computing
(Overview)

1. What's wrong with this code ?

```
int main() {  
    int* p = NULL;  
  
    {  
        int a;  
        p = &a;  
    }  
  
    *p = 2;  
    return 0;  
}
```

A) p should be defined after a

B) a should be initialized

C) p should not be used beyond the block parenthesis

D) Nothing is wrong

1. What's wrong with this code ?

```
int main() {  
    int* p = NULL;  
  
    {  
        int a;  
        p = &a;  
    }  
  
    *p = 2;  
    return 0;  
}
```

A) p should be defined after a

B) A should be initialized

C) p should not be used beyond the block parenthesis

D) Nothing is wrong

« Dangling » Pointer

```
int main() {  
    int* p = NULL;  
    {  
        int a;  
        p = &a;  
    }  
    *p = 2;  
    return 0;  
}
```

- The pointer p **dangles** because it points to the **variable a** that ceased to exist after the **block ended**. When dereferencing a dangling pointer (*p=2), anything can happen including an error segfault.

2. What's wrong with this code ?

```
int* ptr = NULL;  
delete ptr;
```

- A) p should be defined after a
- B) a should be initialized
- C) p should not be used beyond the block parenthesis
- D) Nothing is wrong

2. What's wrong with this code ?

```
int* ptr = NULL;  
delete ptr;
```

- A) p should be defined after a
- B) a should be initialized
- C) p should not be used beyond the block parenthesis
- D) **Nothing is wrong**

2. What's wrong with this code ?

```
int* ptr = NULL;  
delete ptr;
```

Explanation: Nothing prevents you to delete a pointer that is already NULL.

- A) p should be defined after a
- B) a should be initialized
- C) p should not be used beyond the block parenthesis
- D) **Nothing is wrong**

3. If this Code is compiled in C and C++ ?

```
#include<stdio.h>
struct STRUCT
{
private:
int a;
};
int main()
{
printf("%d\n", (int)sizeof(struct
STRUCT));
return 0;
}
```

- A) program runs fine and both prints output "HELLO STRUCTURE"
- B) program gives an error in case of C but runs perfectly in case of C++
- C) program gives an error in case of C++ but runs perfectly in case of C
- D) program gives an error in case of both C and C++

3. If this Code is compiled in C and C++ ?

```
#include<stdio.h>
struct STRUCT
{
private:
int a;
};
int main()
{
printf("%d\n", (int)sizeof(struct
STRUCT));
return 0;
}
```

- A) The program runs fine and both prints output "HELLO STRUCTURE"
- B) **The program gives an error in case of C but runs perfectly in case of C++**
- C) The program gives an error in case of C++ but runs perfectly in case of C
- D) The program gives an error in case of both C and C++

3. If this Code is compiled in C and C++ ?

```
#include<stdio.h>

struct STRUCT
{
private:
int a;
};

int main()
{
printf("%d\n", (int)sizeof(struct
STRUCT));
return 0;
}
```

A)Explanation :

B)Access specifiers like private, protected and the public are used as from OOPs concepts. C is not an Object Oriented language, access specifiers are not defined in C whereas they are in C++. IN a struct object all attributes are « public » by default and cannot be hidden from outside the structure

4. What does this code means ?

```
int(*p[5])();
```

- A) p is a pointer to a function
- B) p is an array of pointer to function
- C) p is pointer to a function which return type is an array
- D) p is a pointer to array of functions

4. What does this code means ?

```
int(*p[5])();
```

A) p is a pointer to a function

B) **p is an array of pointer to function**

C) p is pointer to a function which return type is an array

D) p is a pointer to array of functions

4. What does this code means ?

```
int(*p[5])();
```

Explanation: In the above declaration the variable p is the array, not the pointer.

A) p is a pointer to a function

B) **p is an array of pointer to function**

C) p is pointer to a function which return type is an array

D) p is a pointer to array of functions

5. C vs C++ behavior

Which of the following
type is provided by C++
but not C?

- A) int
- B) bool
- C) float
- D) double

5. C vs C++ behavior

Which of the following type is provided by C++ but not C?

Explanation: C++ provides the boolean type to handle true and false values whereas no such type is provided in C.

- A) int
- B) bool**
- C) float
- D) double

6. What is the output of this C++ code?

```
#include <iostream>
using namespace std;
int main()
{
    char* ptr;
    char Str[] = "abcdefg";
    ptr = Str;
    ptr += 5;
    cout << ptr;
    return 0;
}
```

- A) fg
- B) cdef
- C) defg
- D) abcd

6. What is the output of this C++ code?

```
#include <iostream>
using namespace std;
int main()
{
    char* ptr;
    char Str[] = "abcdefg";
    ptr = Str;
    ptr += 5;
    cout << ptr;
    return 0;
}
```

- A) fg
- B) cdef
- C) defg
- D) abcd

7. Null Pointer

Which of the following does not initialize ptr to null (assuming variable declaration of a as `int a=0;`)?

- A) `int *ptr = &a;`
- B) `int *ptr = &a - &a;`
- C) `int *ptr = a - a;`
- D) All of the mentioned

7. Null Pointer

`int *ptr = &a;` does not initialize the pointer variable `ptr` to NULL or zero. Instead, it assigns `ptr` the memory address of the variable `a`. While the value of `a` can be zero (0), its address cannot be zero. Therefore, `ptr` holds a valid (non-null) memory address pointing to `a`.

- A) `int *ptr = &a;`
- B) `int *ptr = &a - &a;`
- C) `int *ptr = a - a;`
- D) All of the mentioned

8. What is the output ?

```
#include <stdio.h>
int main()
{
    int x = 1;
    int y = x == 1 ? getchar() : 2;
    printf("%d\n", y);
}
```

- A) Compile time error
- B)
- C) 1
- D) Ascii value returned by getchar()
- E) 2

8. What is the output ?

variable = (condition) ? expressionTrue : expressionFalse;

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 1;
```

```
    int y = x == 1 ? getchar() : 2;
```

```
    printf("%d\n", y);
```

```
}
```

A) Compile time error

B)

C) 1

D) Ascii value returned by getchar()

E) 2

8. What is the output ?

```
#include <stdio.h>
int main()
{
    int x = 1;
    int y = x == 1 ? getchar() : 2;
    printf("%d\n", y);
}
```

A) Compile time error

B)

C) 1

D) **Ascii value returned by getchar()**

E) 2

9. What is the output ?

```
#include <iostream>
using namespace std;
int main()
{
    int arr[] = { 4, 5, 9, 13 };
    int* p = (arr + 1);
    cout << *(++p);
    return 0;
}
```

A) 4

B)

C) 5

D) 9

E) 10

9. What is the output ?

```
#include <iostream>
using namespace std;
int main()
{
    int arr[] = { 4, 5, 9, 13 };
    int* p = (arr + 1);
    cout << *(++p);
    return 0;
}
```

A) 4

B)

C) 5

D) 9

E) 10

10. What is the output?

```
#include <iostream>
#include <string>
using namespace std;
class A
{
    int a;

public:
    virtual void func() = 0;
};
class B : public A
{
public:
    void func() {
        cout << "Class B" << endl;
    }
};
```

```
int main(int argc, char const*
argv[])
{
    B b;
    b.func();
    return 0;
}
```

- A) Class B
- B) Error
- C) Segmentation fault
- D) No output

10. What is the output?

```
#include <iostream>
#include <string>
using namespace std;
class A
{
    int a;
public:
    virtual void func() = 0;
};
class B : public A
{
public:
    void func() {
        cout << "Class B" << endl;
    }
};
```

```
int main(int argc, char const*
argv[])
{
    B b;
    b.func();
    return 0;
}
```

- A) Class B
- B) Error
- C) Segmentation fault
- D) No output

11. What is the output?

```
vector<int> numbers = { 1, 7, 3, 5, 9, 2 };  
sort(numbers.begin(), numbers.end());  
for (int num : numbers) {  
    cout << num << " "; }  
}
```

- 1 3 5 7 9 2
- 9 7 5 3 2 1
- 1 2 3 5 7 9
- Random order

11. What is the output?

```
vector<int> numbers = { 1, 7, 3, 5, 9, 2 };  
sort(numbers.begin(), numbers.end());  
for (int num : numbers) {  
    cout << num << " "; }  
}
```

- 1 3 5 7 9 2
- 9 7 5 3 2 1
- **1 2 3 5 7 9**
- Random order

12. What is the output?

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> myvector;
    int sum = 0;
    myvector.push_back(100);
    myvector.push_back(200);
    myvector.push_back(300);
    while (!myvector.empty())
    {
        sum += myvector.back();
        myvector.pop_back();
    }
    cout << sum << '\n';
    return 0;
}
```

A) 500

B) 600

C) 700

D) Error

12. What is the output?

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> myvector;
    int sum = 0;
    myvector.push_back(100);
    myvector.push_back(200);
    myvector.push_back(300);
    while (!myvector.empty())
    {
        sum += myvector.back();
        myvector.pop_back();
    }
    cout << sum << '\n';
    return 0;
}
```

A) 500

B) 600

C) 700

D) Error

13. What is the output?

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    do
```

```
        printf("In while loop ");
```

```
    while (0);
```

```
    printf("After loop\n");
```

```
}
```

A) In while loop

B) In while loop After loop

C) After loop

D) Infinite loop

13. What is the output?

```
#include <stdio.h>

int main()
{
    do

        printf("In while loop ");

    while (0);

    printf("After loop\n");
}
```

A) In while loop

B) In while loop After loop

C) After loop

D) Infinite loop

E) do-while loop executes its body at least once before checking the condition. So, 'In while loop' is printed. Since the condition is false, the loop terminates, and the program proceeds to execute printf("after loop\n").]

14. What is the output?

```
#include <stdio.h>
int main()
{
    int a = 0, i = 0, b;
    for (i = 0; i < 5; i++)
    {
        a++;
        if (i == 2)
            break;
    }
    printf("%d", a);
}
```

A) 1

B) 2

C) 3

D) 4

14. What is the output?

```
#include <stdio.h>
int main()
{
    int a = 0, i = 0, b;
    for (i = 0; i < 5; i++)
    {
        a++;
        if (i == 2)
            break;
    }
    printf("%d", a);
}
```

- A) 1
- B) 2
- C) 3
- D) 4

E) When the condition `i == 3` is met, the break statement terminates the loop, skipping the remaining iterations. As a result, the variable `a` is updated 4 times, and its final value is 4]

Projects Hands-on

- 1) Hands-on on Your Projects
- 2) Rehearsal exercises
- 3) **Course : Multi-task & High Performance Computing (Overview)**

Part 1 - Multi-tasking

Multi-tasking : doing
several things
apparently at the
same time



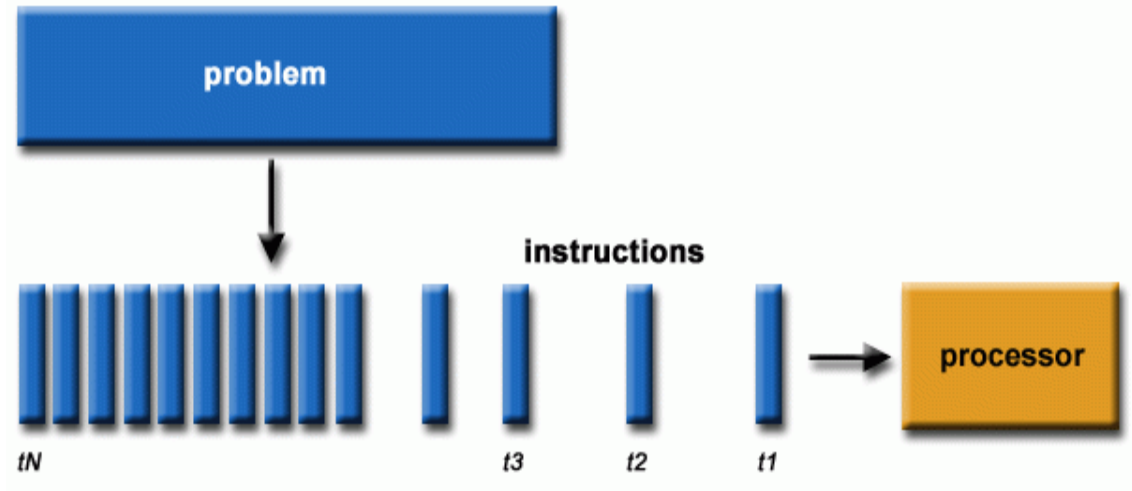
Part 1 - Multi-tasking

Doing several
things apparently
at the same time

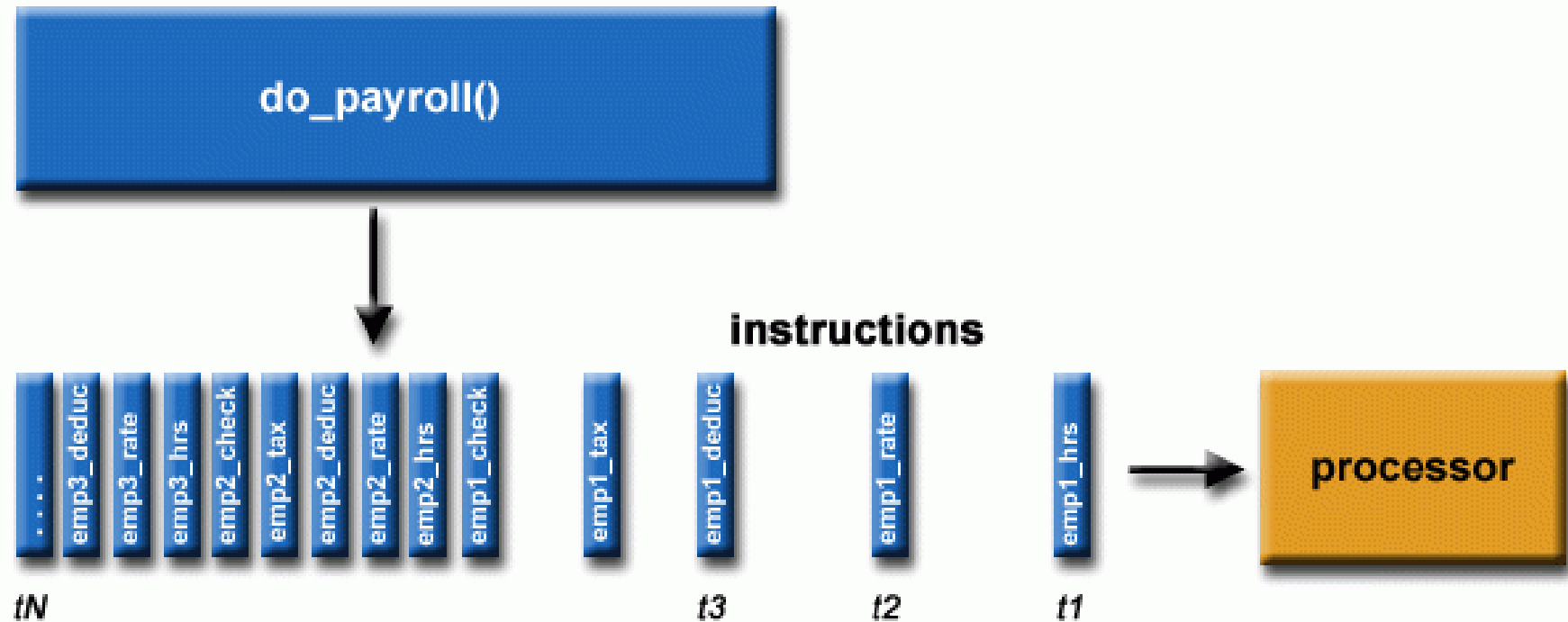


Serial Computing

- A problem statement is broken into discrete instructions.
- Instructions are executed one by one.
- Only one instruction is executed at any moment of time.

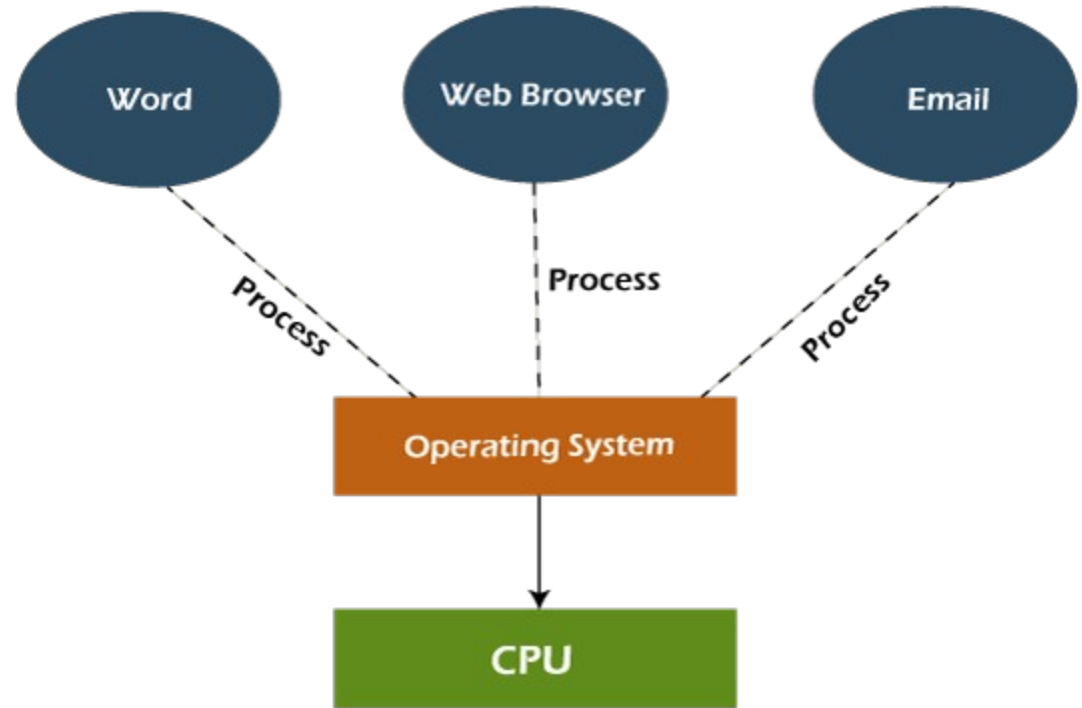


Serial Computing - example



Multi-tasking

- Multitasking Operating System : allows several applications to run « simultaneously ». Enables user to execute multiple tasks at the same time
- gives the perception of two or more tasks/processes running simultaneously.
- Done by dividing resources amongst tasks/processes and switching between the them while they are executing over and over again.
- CPU processes only one task at a time, but in Multitasking the switching is so fast that it looks like the CPU is executing multiple processes simultaneously.

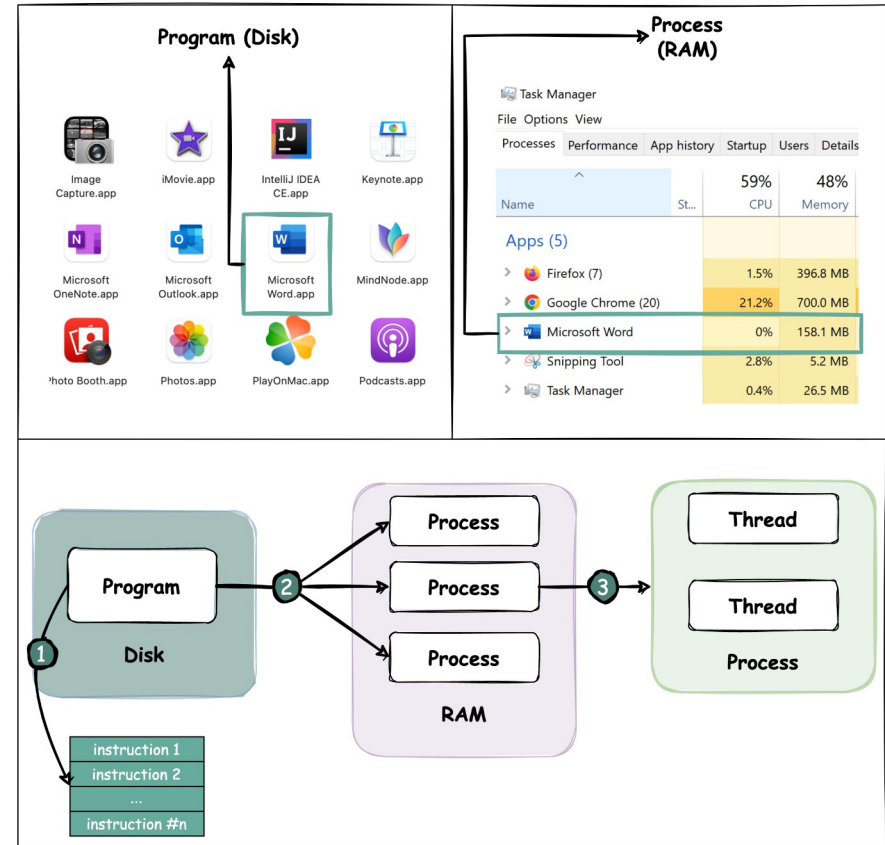


Program vs Process

- Program : executable file on disk.
- Process : created by the Operating System to execute a program.
- Runs in **its own address space**

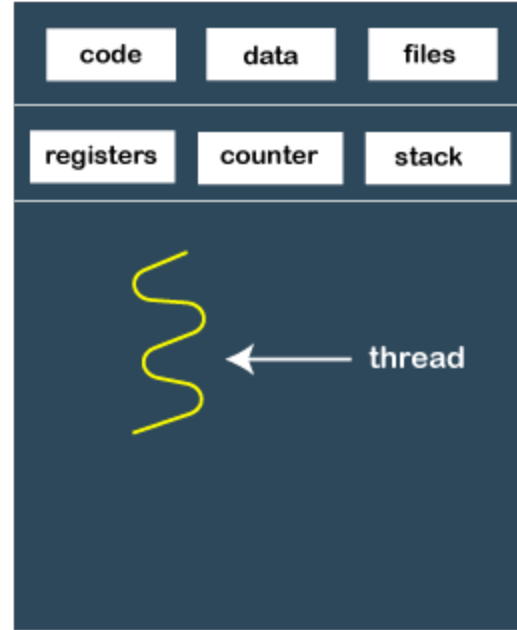
Program vs Process vs Thread

 blog.bytebytego.com

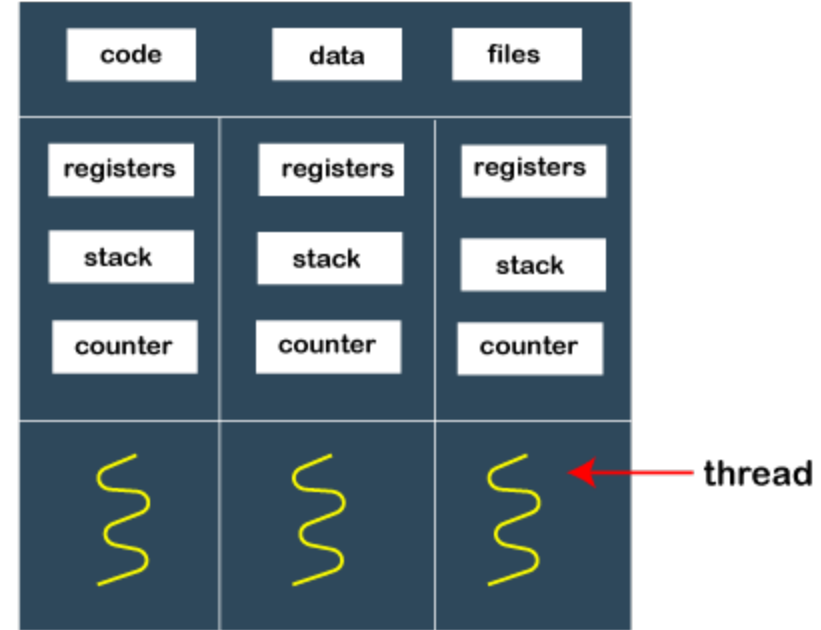


Thread vs Process

- Process : independent program or application that runs in **its own address space**
- Thread : smallest unit of execution **within a process** ; shares the same address space as the process.
- Each process has its own virtual memory space, while threads share the same virtual memory space.



Single-threaded process



Multi-threaded process

Multi-tasking : Process or Thread-based

Process-Based	Thread-Based
two or more processes/programs can run concurrently.	two or more threads can run concurrently.
Each process requires its own address space	Threads share the same address space
Process to Process communication is expensive.	Thread to Thread communication is not expensive.
Example:/ listen to music and browse the internet at the same time. The processes are the music player and browser .	Example: browser navigates through web pages and at the same time download a file. navigation is one thread, downloading another thread.

Process to Process Communication

- **conflict** between process are **nearly impossible** if you don't need them to communicate
- On a single computer or across vast networks, programs might need to talk to each other. Such exchange of information, aka **process-to-process communication** (IPC), must be coded as the foundation of any system where multiple programs work together

Thread to Thread Communication

- By construction, **conflict** between threads are **POSSIBLE**. Threads can directly communicate with each other as **they share the same address space**.
- two or more threads may try to change a common data at the same time.

Multi-threading

- allows a single process to create and manage multiple threads of execution within its own memory space. Each thread operates **independently**, performing separate tasks **concurrently** within the same application.
- Concurrency: Multithreading enables **concurrent** execution of tasks, allowing applications to handle multiple operations simultaneously. For example, a web browser can render a webpage, download files, and handle user inputs all at the same time using different threads.

Multi-threading

- Threads within same process share resources such as **memory, file handles**,...
- Facilitates efficient communication and data sharing between threads but
- **requires careful synchronization** to prevent issues like **race conditions**.

Data Race between Threads

Race condition is a situation where two or more threads happen to change a common data at the same time. If there is a race condition then we have to protect it and the protected section is called critical section/region

Managing conflicts - MUTEX

- Multiple threads may try to access and modify such resources simultaneously. This can result in **data races**, where the result of the program becomes unpredictable.
- to ensure that only one thread or process can access a **shared resource** at a time, a **mutex** - « **Mutual Exclusion** » - is a **lock** that can be acquired or released by a thread or process to control the access to the resource.

Multi-threading Example : « Bounce »

- « **Bounce** » : simple multi-thread program **creates a new thread** each time « **a** » or « **A** » is typed.
- Each thread bounces a letter of different color around the screen. Up to 32 threads can be created.
- Program stops when « **q** » or « **Q** » is typed.

Managing conflicts - MUTEX

// Mutex Lock Example

```
#include <pthread.h>
```

```
pthread_mutex_t count_mutex;
```

```
long long count; // SHARED VARIABLE
```

```
Void increment_count(){
```

```
    pthread_mutex_lock(&count_mutex);
```

```
    count = count + 1;
```

```
    pthread_mutex_unlock(&count_mutex);
```

```
}
```

```
long long get_count(){
```

```
    long long c;
```

```
    pthread_mutex_lock(&count_mutex);
```

```
    c = count;
```

```
    pthread_mutex_unlock(&count_mutex);
```

```
    return (c);
```

```
}
```

- **increment_count()** uses the mutex lock to ensure an atomic update of the shared variable.
- **get_count()** uses the mutex lock to guarantee that the 64-bit quantity count is read atomically.

Other Sync mechanisms - Semaphore

- **Semaphores** (sometimes referred to as counting semaphores) are used to control access to shared resources. A semaphore is an intelligent counter.
- Every semaphore has a current count, which is greater than or equal to 0.
- Any thread can **decrement** the count to lock the semaphore (also called waiting on the semaphore). Attempting to decrement the count past 0 causes the thread that is calling to wait for another thread to unlock the semaphore.
- Any thread can **increment** the count to unlock the semaphore (this is also called posting the semaphore). Posting a semaphore might wake up a waiting thread if there is one present.
- In their simplest form (with an initial count of 1), semaphores can be thought of as a mutual exclusion (mutex). The important distinction between semaphores and mutexes is the concept of ownership. **No ownership is associated with a semaphore**. Unlike mutexes, it is possible for a thread that never waited for (locked) the semaphore to post (unlock) the semaphore. This can cause unpredictable application behavior. You must avoid this if possible.

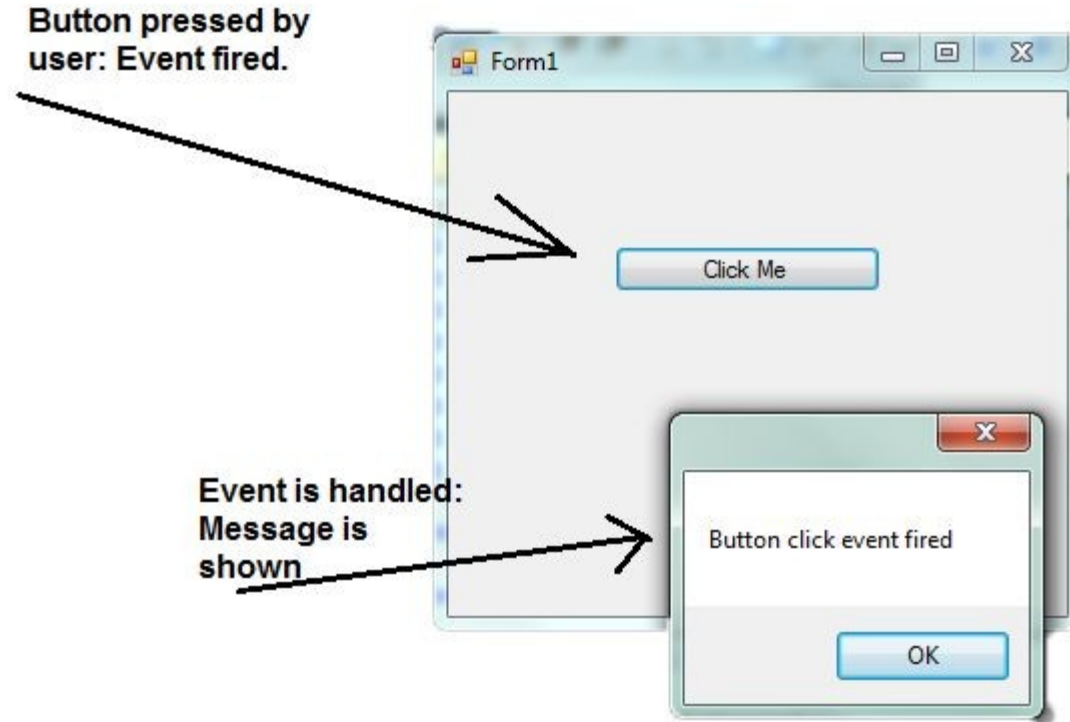
Event handling - Signals

- A **signal** is an **asynchronous notification** sent to a process or to a specific thread within the same process to notify it of an event. Common uses of signals are to interrupt, suspend, terminate or kill a process.
- **Signals** originated in 1970s Bell Labs Unix and were later specified in the POSIX standard.



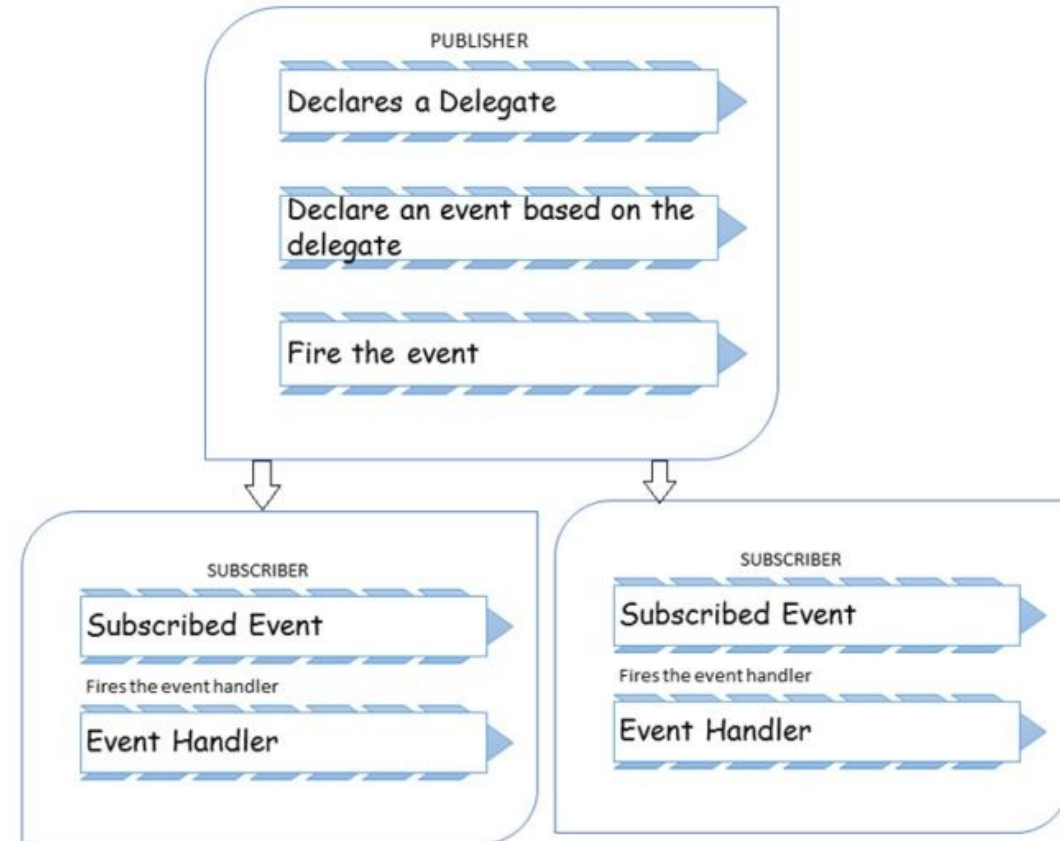
Event handling - Signals

- A **signal** is an asynchronous notification sent to a process or to a specific thread within the same process to notify it of an event. Common uses of signals are to interrupt, suspend, terminate or kill a process.
- **Signals** originated in 1970s Bell Labs Unix and were later specified in the POSIX standard.



Publisher – Subscriber Model

- **Events** are part of a class and are « fired » by this class. Other classes can « subscribe » to these events.
- A **publisher** contains the definition of the event and the delegate (event manager).
- A **subscriber** is an object that wants to accept the event and provide a handler to the event.



Exercise : bounce program

- Get code provided in multi-thread.zip
- Read it
- Compile it
- Run it
- Comment it

Multi-Core Hardware

- capability of a CPU (Central Processing Unit) to have more than one processing unit, or **core**, within a single chip. Each core can execute instructions independently, allowing **true parallelism** in processing tasks.
- **Parallel Processing**: With multiple cores, a CPU can perform multiple operations **simultaneously**. For instance, a **quad-core processor** can **handle four separate tasks** at the same time, each on its own core, leading to significant performance improvements for multi-threaded applications.

Evolutions of Microprocessors

Year	Architecture	Name	Clock Speed	Nb of Transistors	Instructions per second
1971	4 bits	INTEL 4004/4040	740 kHz	2300	60000
1972	8 bits	INTEL 8008	500 kHz	3500	50000
1974	8 bits	INTEL 8080	2 MHz	6000	10 times faster than 8008
1976	8 bits (16-bits bus)	INTEL 8085	3 MHz	6500	769230
1978	16 bits	INTEL 8086/8088 (First IBM PC 1981)	4.77 MHz, 8 MHz, 10 MHz	29000	2.5 million
1982	16 bits	INTEL 80286	8 MHz	134000	4 million

Evolutions of Microprocessors

Year	Architecture	Name	Clock Speed	Nb of Transistors	Instructions per second
1986	32 bits	INTEL 80386	16 - 33 MHz	275000	
1993	32 bits	Pentium	66 MHz		Cache memory
2006	64 bits	INTEL Core 2 FIRST MULTI-CORE	1.2 to 3 GHz	291 million	64 KB of L1 cache per core 4 MB of L2 cache
2007-2010	64 bits	Intel i3, i5, i7 8 cores – 16 threads	2.2-3.3 GHz, 2.4-3.6GHz 2.93-3.33 GHz		

Multi-Core Hardware

- Independent Execution: Each core operates independently, running its own threads or processes without interfering with others. This independence allows for better scalability and performance, **especially in multi-core optimized software.**
- Increased Throughput: Multi-cores enhance the overall throughput of a system by distributing the workload across several cores, reducing time required to complete tasks compared to a single-core processor.

Multi-threading on Multi-Core CPUs

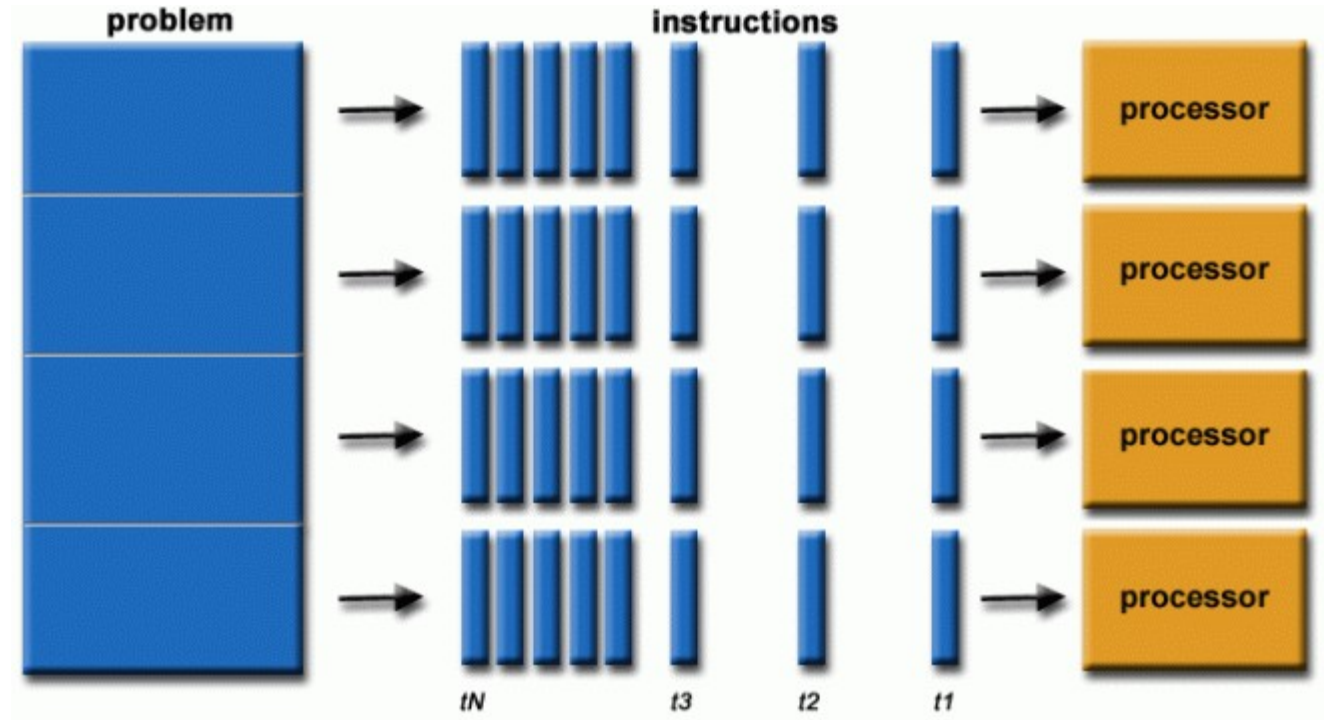
Multithreading is a form of parallelization or dividing up work for simultaneous processing. Instead of giving a large workload to a single core, threaded programs split the work into **multiple software threads**. These threads are processed in parallel by **different CPU cores** to save time.

Multi-threading on Multi-Core CPUs

- In a single-processor system, multiple threads execute , one after the other or wait until one thread finishes or is preempted by the OS , depending on the thread priority and the OS policy. This gives the **illusion** that they run simultaneous , relative to the required application response time of the User space application.
- In a multi-processor system , multiple threads execute , simultaneously on different **cores**. If there are two threads and two cores , each thread runs on an individual core.
- **ACTUAL Parallel Computing** is achieved only with **multi-threading** on **multi-core** processor

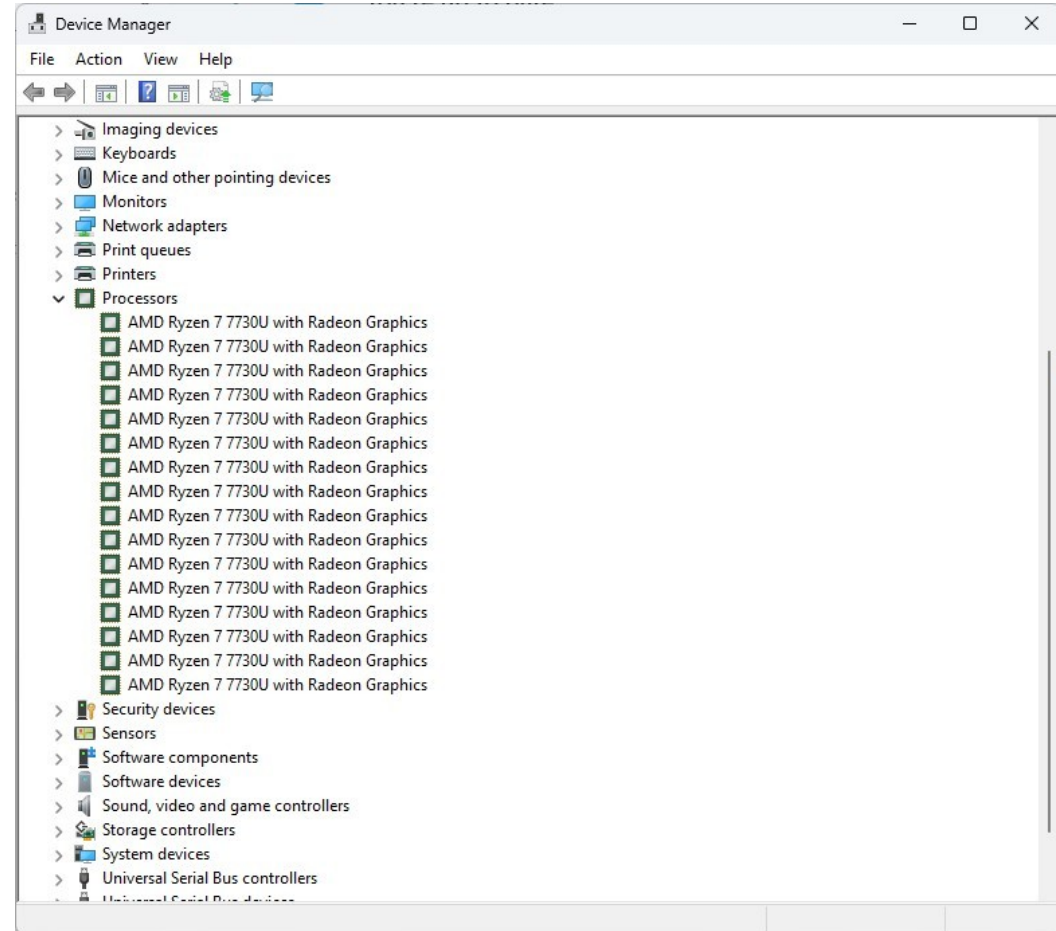
Part 2 - Parallel Computing

- Simultaneous use of multiple computing resources to solve a computational problem
- Problem is broken into discrete parts that can be solved **concurrently**. Each part is further broken down to a series of instructions.



Multi-core processor (MCP)

- A microprocessor on a single integrated circuit (IC) with **two or more** separate central processing units (**CPUs**), called **cores** to emphasize their multiplicity (ex : dual-core or quad-core).
- Each core reads and executes program ordinary CPU instructions (such as add, move data, and branch).
- MCP can run instructions on separate cores **at the same time**, increasing overall speed for programs supporting multithreading or other parallel computing techniques.
- Cores integrated onto a chip multiprocessor (**CMP**)
- As of 2024, nearly all new PCs are multi-core.



Parallel Computing applications

- Real-time simulation of systems.
- Scientific and Engineering applications.
- Heavy Computations, modelisations, simulations
- Advanced graphics, augmented and virtual reality.



Parallel Computing Libraries

process.h

In C++, the header file **process.h** is part of the C standard library that provides functions for process control. `process.h` is not part of the C++ standard library and is **typically specific to certain compilers or platforms, such as Microsoft Visual C++**. Its primary use is in environments where you need to manage processes, such as creating and terminating processes, and handling process-related information.

OpenMP

OpenMP (Open Multi-Processing) is an application programming interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran, on many platforms, instruction-set architectures and operating systems, including Solaris, AIX, FreeBSD, HP-UX, Linux, macOS, and Windows. It consists of a set of compiler directives, library routines, and environment variables that influence run-time behavior.

OpenMP is managed by the nonprofit technology consortium OpenMP Architecture Review Board (or OpenMP ARB), jointly defined by a broad swath of leading computer hardware and software vendors, including Arm, AMD, IBM, Intel, Cray, HP, Fujitsu, Nvidia, NEC, Red Hat, Texas Instruments, and Oracle Corporation.

Intel TBB - Threading Building Blocks

Threading Building Blocks (**oneTBB**; formerly Threading Building Blocks or TBB) is a C++ template library developed by Intel for parallel programming on multi-core processors. Using TBB, a computation is broken down into tasks that can run in parallel. The library manages and schedules threads to execute these tasks.

A **oneTBB** program creates, synchronizes, and destroys graphs of dependent tasks according to algorithms, i.e. high-level parallel programming paradigms (a.k.a. Algorithmic Skeletons). Tasks are executed respecting graph dependencies. This approach groups TBB in a family of techniques for parallel programming aiming to **decouple** the programming from the **particulars** of the underlying machine.

Part 3 - HPC Computing

High performance computing (HPC) is the practice of **aggregating computing resources** to gain performance greater than that of a single workstation, server, or computer. HPC can take the form of custom-built supercomputers or groups of individual computers called **clusters**.



HPC Computing

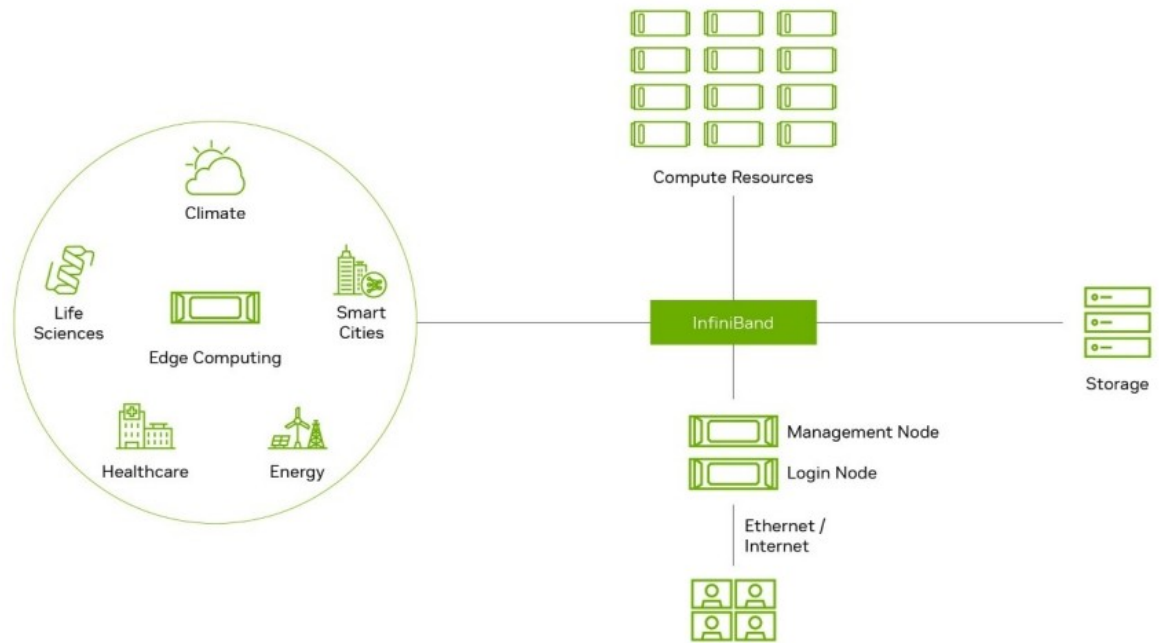
HPC computer systems are characterized by their high-speed processing power, high-performance networks, and large-memory capacity, generating the capability to perform **massive amounts of parallel processing**. A supercomputer is a type of HPC computer that is highly advanced and provides **immense computational power** and **speed**, making it a key component of high-performance computing systems.

In recent years, HPC has evolved from a tool focused on simulation-based scientific investigation to a dual role running simulation and machine learning (ML). This increase in scope for HPC systems has gained momentum because the combination of physics-based simulation and ML has compressed the time to scientific insight for fields such as climate modeling, drug discovery, protein folding, and computational fluid dynamics (CFD).

HPC Computing

The basic system architecture of a supercomputer.

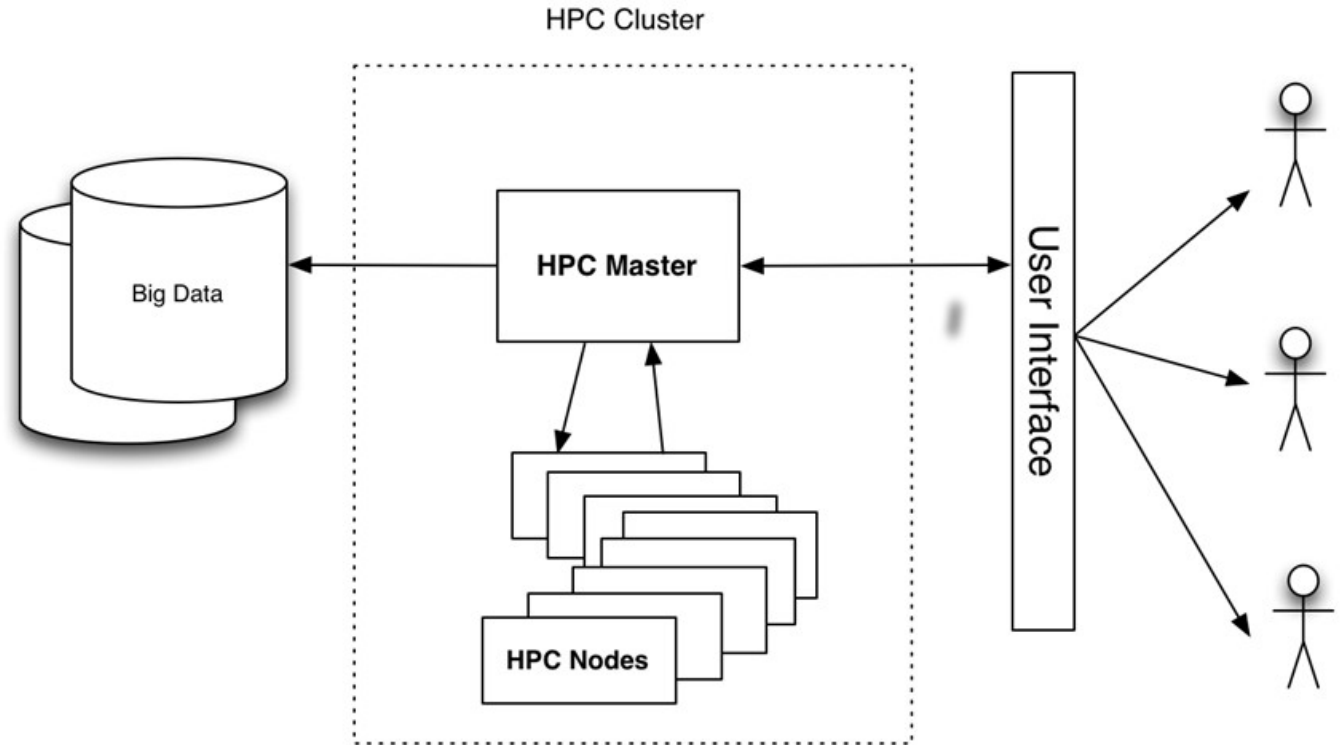
One key enabler driving this evolution of HPC and ML is the development of **graphics processing unit (GPU)** technology. GPUs are specialized computer chips **designed to process large amounts of data in parallel**, making them ideal for some HPC, and are currently the standard for ML/AI computations. The combination of high-performance GPUs with software optimizations has enabled HPC systems to perform complex simulations and computations much faster than traditional computing systems.



HPC Computing Terms

Cloud : generic term used to refer to remote computing resources of any kind. Can refer to machines serving websites, providing shared storage, webservices (e-mail or social media), as well as more traditional “compute” resources.

•An **HPC system** on the other hand, is a term used to describe a **network of computers**. The computers in a cluster typically share a **common purpose**, and are used to accomplish tasks that might otherwise be too big for any one computer.



HPC Computing Domains

- Molecular dynamics simulation
- Computational fluid dynamics
- Climate modeling
- Computational chemistry
- Structural mechanics and engineering

Electromagnetic simulation

Seismic imaging and analysis

Materials science and engineering

Astrophysical simulation

Machine learning and data analysis

HPC Computing Domains

- Independent Execution: Each core operates independently, running its own threads or processes without interfering with others. This independence allows for better scalability and performance, **especially in multi-core optimized software.**
- Increased Throughput: Multi-cores enhance the overall throughput of a system by distributing the workload across several cores, reducing time required to complete tasks compared to a single-core processor.

Electromagnetic simulation

Seismic imaging and analysis

Materials science and engineering

Astrophysical simulation

Machine learning and data analysis

How does HPC Computing work

Combining the computational power of multiple computers to perform large-scale tasks that would be infeasible on a single machine. Here is how :

Cluster Configuration: An HPC cluster is made up of multiple computers, or nodes, that are connected by a high-speed network. Each node is equipped with one or more processors, memory, and storage.

Task Parallelization: The computational work is divided into smaller, independent tasks that can be run simultaneously on different nodes in the cluster. This is known as task parallelization.

Data Distribution: The data required for the computation is distributed among the nodes, so that each node has a portion of the data to work on.

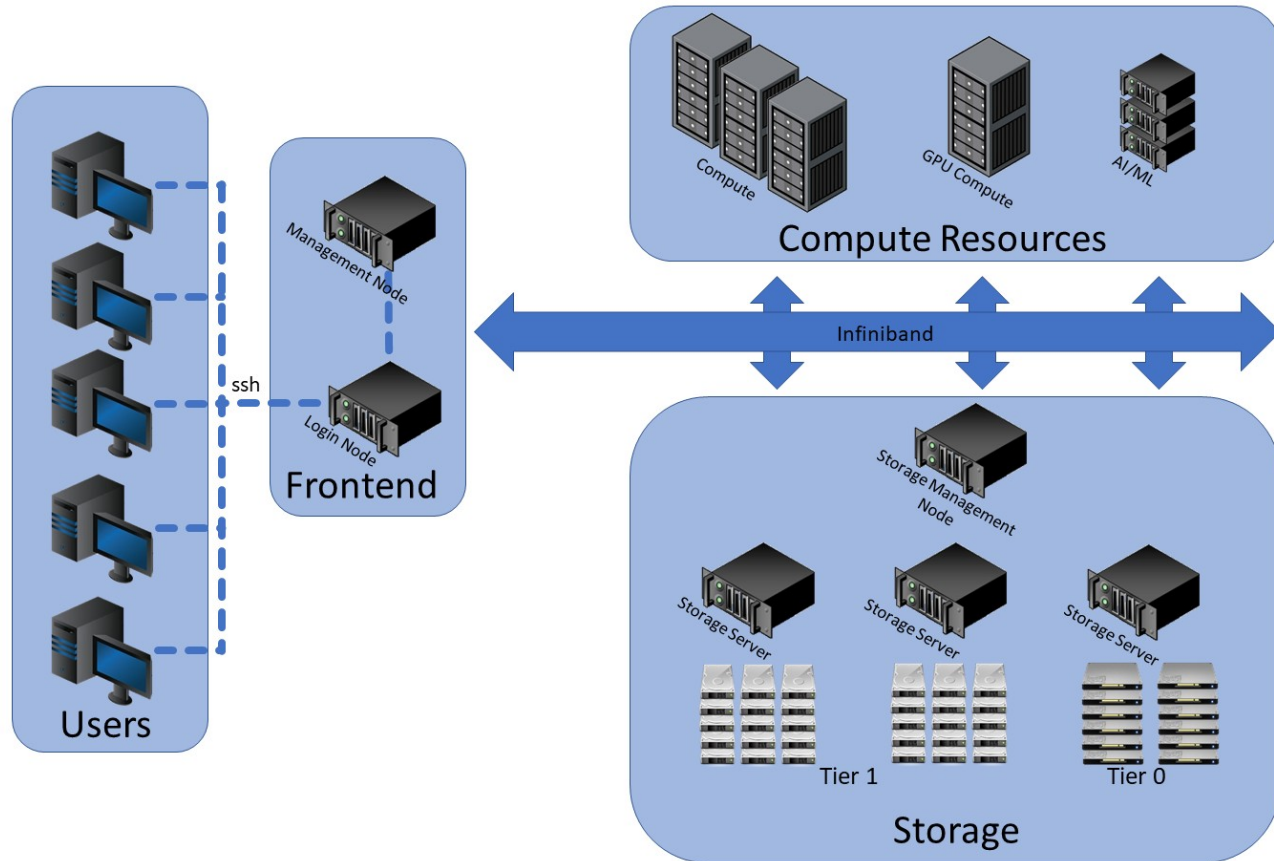
How does HPC Computing work

Computation: Each node performs its portion of the computation in parallel, results are shared and ultimately integrated until work complete

Monitoring and Control: The cluster includes software tools that monitor the performance of the nodes and control the distribution of tasks and data. This helps ensure that the computation runs efficiently and effectively.

Output: is the result of the combined computation performed by all the nodes in cluster. The output is generally saved to a large parallel file system and/or rendered graphically into images or other visual depictions to facilitate discovery, understanding, and communication.

By harnessing the collective power of many computers, HPC enables large-scale simulation, data analysis, and other compute-intensive tasks to be completed in a fraction of the time it would take on a single machine.



HPC Computing Performance

FLOPS =

Floating
Operations
Per Second

Name	Unit	Value
kiloFLOPS	kFLOPS	10^3
megaFLOPS	MFLOPS	10^6
gigaFLOPS	GFLOPS	10^9
teraFLOPS	TFLOPS	10^{12}
petaFLOPS	PFLOPS	10^{15}
exaFLOPS	EFLOPS	10^{18}
zettaFLOPS	ZFLOPS	10^{21}
yottaFLOPS	YFLOPS	10^{24}
ronnaFLOPS	RFLOPS	10^{27}
quettaFLOPS	QFLOPS	10^{30}

Top 500 Super Computers Ranking

Nov. 18, 2024

El Capitan has the top spot and is officially the **third system to reach exascale computing** after **Frontier** and **Aurora**.

Name	Spec	Speed
El Capitan	HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE	1.742 EFlop/s
Frontier	HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE	1.353 Eflop/s
Aurora	HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel	1.012 Exaflop/s

HPC Computing Domains

Speed and Efficiency: HPC systems can perform complex calculations much faster than traditional computers, allowing researchers and engineers to tackle large-scale problems that would be infeasible with conventional computing resources.

Scientific Discovery: HPC is critical for many scientific disciplines, including climate modeling, molecular dynamics, and computational fluid dynamics. It allows researchers to simulate complex systems and processes, leading to new insights and discoveries.

Product Design and Optimization: HPC is widely used in industries such as aerospace, automotive, and energy to simulate and optimize the design of products, processes, and materials, improving their performance and reducing development time.

HPC Computing Domains

Data Analysis: HPC is also essential for analyzing large datasets, such as those generated by observational studies, simulations, or experiments. It enables researchers to identify patterns and correlations in the data that would be difficult to detect using traditional computing resources.

•**Healthcare:** HPC is increasingly being used in healthcare to develop new treatments and therapies, including personalized medicine, drug discovery, and molecular modeling.

HPC has revolutionized the way research and engineering are conducted and has had a profound impact on many aspects of our lives, from improving the efficiency of industrial processes to disaster response and mitigation to furthering our understanding of the world around us.